深度之眼
deepshare.net

PINN疑难杂症诊断：伪解、不收敛问题全解析

主讲人：择木老师

深度之眼

2025 年 2 月 27 日

目录

深度之眼
deepshare.net

PINN硬约束

PINN求解多尺度问题

Section 1

PINN硬约束

硬约束边界条件简介

深度之眼
deepshare.net

在PINN中实现边界条件的两种主要方法：

- **软约束**：通过损失函数强制实现（适用于所有边界条件类型）
- **硬约束**：通过修改网络架构严格满足边界条件

硬约束方法的优势：

- 精确满足边界条件（不是近似满足）
- 减少计算成本（减少损失项）
- 实现相对简单（对于特定类型的边界条件）

两种可以实施的硬约束：

- Dirichlet边界条件
- 周期性边界条件

硬约束Dirichlet边界条件

深度之眼 deepshare.net

Dirichlet边界条件: $u_i(\mathbf{x}) = g(\mathbf{x})$ on Γ_D 修改解的形式:

$$\hat{u}_i(\mathbf{x}; \boldsymbol{\theta}_u) = g(\mathbf{x}) + \ell(\mathbf{x})\mathcal{N}(\mathbf{x}; \boldsymbol{\theta}_u)$$

其中$\ell(\mathbf{x})$满足:

$$\begin{cases} \ell(\mathbf{x}) = 0, & \mathbf{x} \in \Gamma_D, \\ \ell(\mathbf{x}) > 0, & \mathbf{x} \in \Omega - \Gamma_D. \end{cases}$$

简单区域的$\ell(\mathbf{x})$选择:

- 区间$[a, b]$: $(x - a)(b - x)$
- 复杂区域: 使用样条函数、三角函数

参考文献: *Physics-informed neural networks with hard constraints for inverse design.*

硬约束周期性边界条件

深度之眼
deepshare.net

对于周期为P的周期函数，可以通过傅里叶级数分解:

$$u_i(\mathbf{x}) = \sum_{k=0}^{\infty} a_k \cos\left(\frac{2\pi k x_j}{P}\right) + b_k \sin\left(\frac{2\pi k x_j}{P}\right)$$

修改神经网络输入:

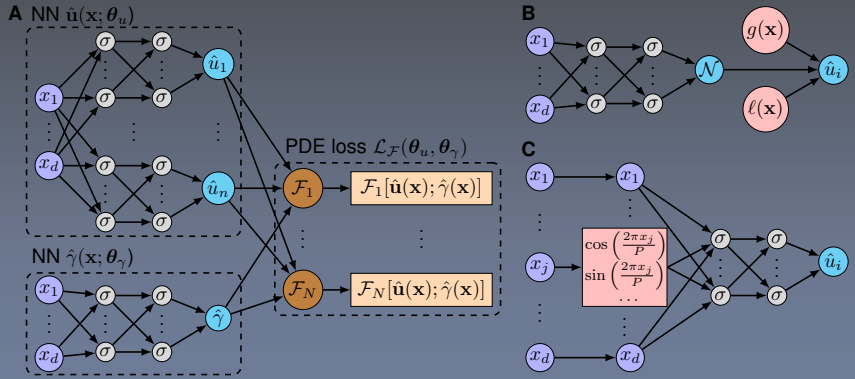
$$\mathcal{N}\left(x_1, \cdots, x_{j-1}, \left[\cos\left(\frac{2\pi x_j}{P}\right), \sin\left(\frac{2\pi x_j}{P}\right)\right], x_{j+1}, \cdots, x_d\right)$$

简化实现:

- 仅使用两个基函数即可
- 神经网络可以通过非线性层自动学习高频分量

PINN硬约束架构

深度之眼
deepshare.net



(A) 标准PINN架构：两个独立神经网络$\hat{\mathbf{u}}(\mathbf{x};\boldsymbol{\theta}_u)$和$\hat{\gamma}(\mathbf{x};\boldsymbol{\theta}_\gamma)$分别近似$\mathbf{u}(\mathbf{x})$和$\gamma(\mathbf{x})$，梯度通过自动微分(AD)计算

(B) 硬约束Dirichlet边界条件：通过修改网络输出实现

(C) 硬约束周期性边界条件：通过修改网络输入实现

波动方程的硬约束设计

深度之眼 deepshare.net

硬约束方法：

- 网络输出重新定义为：

$$\hat{u}'(x,t) = g(x,t) + \ell(x,t)\mathcal{N}(x,t;\boldsymbol{\theta}),$$

具体定义：

- 初始条件：

$$u(x,0) = \sin(\pi x) + \sin(2\pi x), \quad \frac{\partial u(x,0)}{\partial t} = 0,$$

- 边界条件：

$$u(0,t) = 0, \quad u(1,t) = 0,$$

- 选择：

$$g(x,t) = \sin(\pi x) + \sin(2\pi x), \quad \ell(x,t) = \sin(\pi x)t^2.$$

优势：

- 确保网络输出严格满足 ICs 和 BCs，无需通过损失函数优化。
- 提高模型预测的精度与物理一致性。

硬约束实施代码 1/2

深度之眼
deepshare.net

```python
    def forward(self, x, t):
        """
        前向传播，输入x和t，输出预测值u
        使用硬约束方法，满足初始条件和边界条件
        """
        input_combined = torch.cat([x, t], dim=-1)  # 拼接输入 [batch_size, 2]
        out = input_combined
        for layer in self.layers[:-1]:
            out = self.activation(layer(out))
        out = self.layers[-1](out)  # 最后一层无激活函数

        # 使用硬约束满足所有边界条件和初始条件
        # 1. 初始值条件 u(x, 0) = sin(πx) + sin(2πx)
        # 2. 初始导数条件 u_t(x, 0) = 0
        # 3. 边界条件 u(0, t) = 0 和 u(1, t) = 0

        # 硬约束
        out = torch.sin(np.pi*x) + torch.sin(2*np.pi*x) + torch.sin(np.pi*x)*t**2 * out

        return out
```

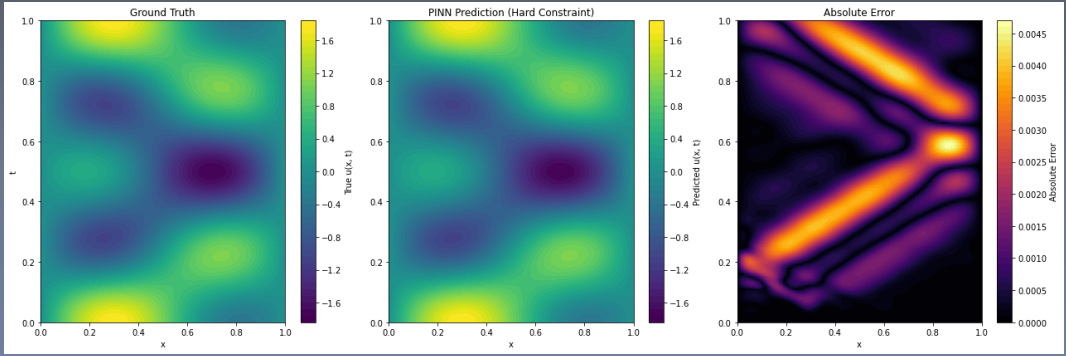
硬约束实施代码 2/2

深度之眼
deepshare.net

```python
    def losses(self, x, t):
        """
        计算PDE残差损失，虽然使用硬约束满足了边界和初始条件，
        但我们仍然计算所有点的PDE残差

        参数:
        x: 训练数据的空间坐标(字典格式)
        t: 训练数据的时间坐标(字典格式)

        返回:
        loss_pde: PDE残差损失
        """
        # 将所有类型的点拼接起来计算PDE残差
        x_combined = torch.cat([x['pde'], x['ic'], x['bc']], dim=0)
        t_combined = torch.cat([t['pde'], t['ic'], t['bc']], dim=0)

        # 计算PDE残差
        _, u_xx, u_tt = self.calculate_derivatives(x_combined, t_combined)
        residual = u_tt - c_true**2 * u_xx
        loss_pde = torch.mean(residual**2)

        # 初始化损失历史(如果尚未初始化)
        if not hasattr(self, 'history'):
            self.history = {
                'loss_pde': []
            }

        # 记录损失
        self.history['loss_pde'].append(loss_pde.item())

        return loss_pde
```

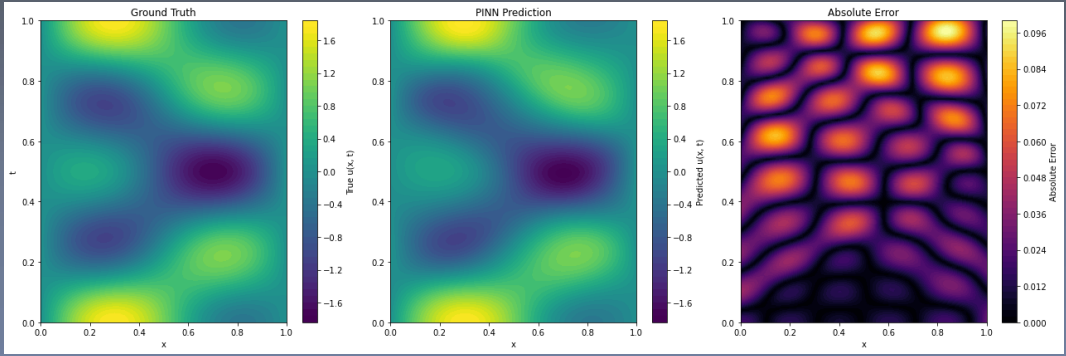
硬约束结果

深度之眼
deepshare.net



软约束结果

深度之眼
deepshare.net



边界条件的硬约束实现难度分析

深度之眼 deepshare.net

并非所有偏微分方程的边界条件都能轻松实现硬约束。边界条件的可实现性取决于其类型和复杂性：

较易实现硬约束的情况

- **简单Dirichlet边界条件**：通过 $\hat{u}(x) = A(x) + F(x, \theta) \cdot B(x)$ 形式可以直接实现
- **周期性边界条件**：可通过特定网络架构或三角函数基函数实现

实现困难或受限的情况

- **复杂Neumann边界条件**：涉及导数约束，实现难度更高
- **混合边界条件**：需要同时满足多种约束条件
- **非线性边界条件**：很难找到通用的参数化形式
- **自由边界/移动边界问题**：边界本身是解的一部分
- **高维复杂几何区域**：边界表达复杂时难以构造合适的参数化

Section 2

PINN求解多尺度问题

一维波动方程正问题

深度之眼 deepshare.net

方程形式：

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0, \ (x,t) \in [0,1]^2$$

初始条件：

$$u(x,0) = \sin(\pi x) + \sin(2\pi x), \quad \frac{\partial u(x,0)}{\partial t} = 0$$

边界条件：

$$u(0,t) = 0, \quad u(1,t) = 0$$

解析解： 根据 d'Alembert 公式，波动方程的解析解为：

$$u(x,t) = \frac{1}{2}\left[f(x+ct) + f(x-ct)\right]$$

其中 $f(x) = \sin(\pi x) + \sin(2\pi x)$。

目标：使用PINN求解$c = 10$工况，使得PINN的解（预测值）与解析解够足接近

PINN波动方程正问题损失函数构建

深度之眼 deepshare.net

PINN 的综合损失函数：

$$\mathcal{L}(\boldsymbol{\theta}) = \lambda_r \mathcal{L}_r(\boldsymbol{\theta}) + \lambda_{i1}\mathcal{L}_{i1}(\boldsymbol{\theta}) + \lambda_{i2}\mathcal{L}_{i2}(\boldsymbol{\theta}) + \lambda_b \mathcal{L}_b(\boldsymbol{\theta})$$

各项定义为：

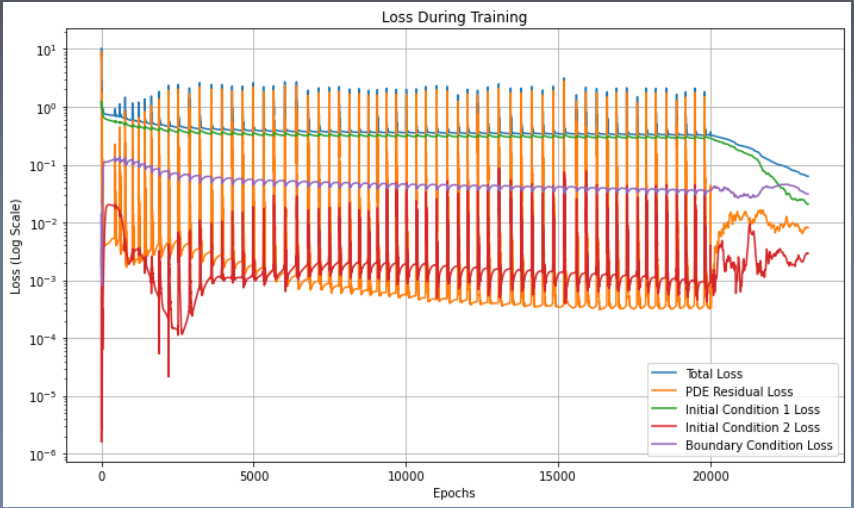
$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{N_r}\sum_{k=1}^{N_r}\left(\frac{\partial^2 \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_r^k, t_r^k)}{\partial t^2} - 10^2 \frac{\partial^2 \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_r^k, t_r^k)}{\partial x^2}\right)^2,$$

$$\mathcal{L}_{i1}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^k, 0) - \sin(\pi x_i^k) - \sin(2\pi x_i^k)\right)^2,$$

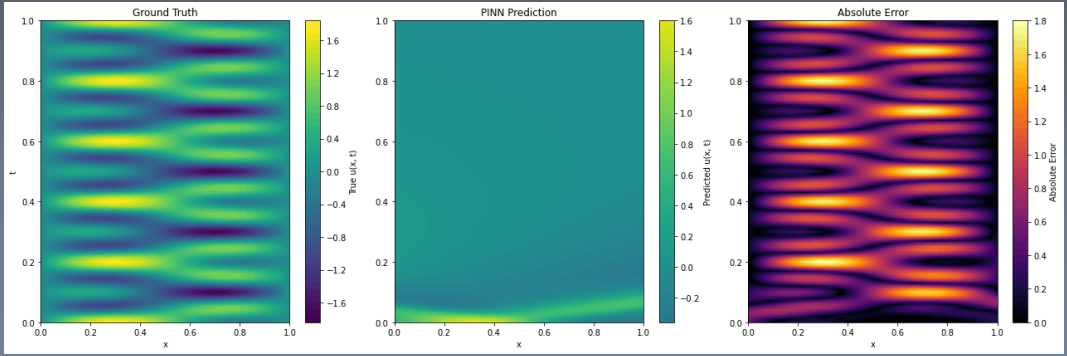
$$\mathcal{L}_{i2}(\boldsymbol{\theta}) = \frac{1}{N_i}\sum_{k=1}^{N_i}\left(\frac{\partial \hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^k, 0)}{\partial t} - 0\right)^2,$$

$$\mathcal{L}_b(\boldsymbol{\theta}) = \frac{1}{N_b}\sum_{k=1}^{N_b}\left(\hat{u}_{\boldsymbol{\theta}}(\boldsymbol{x}_b^k, t_b^k) - 0\right)^2.$$

可视化：损失函数值($\lambda = 1$)

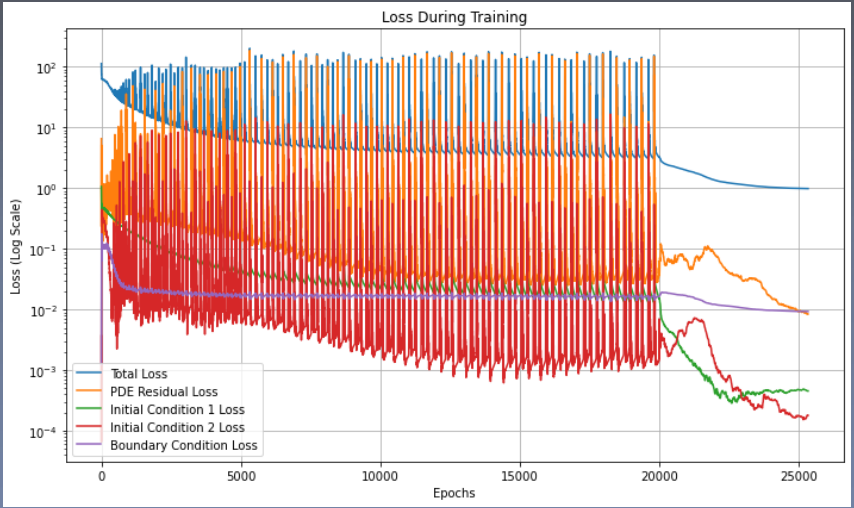


可视化：预测值对比值($\lambda = 1$)



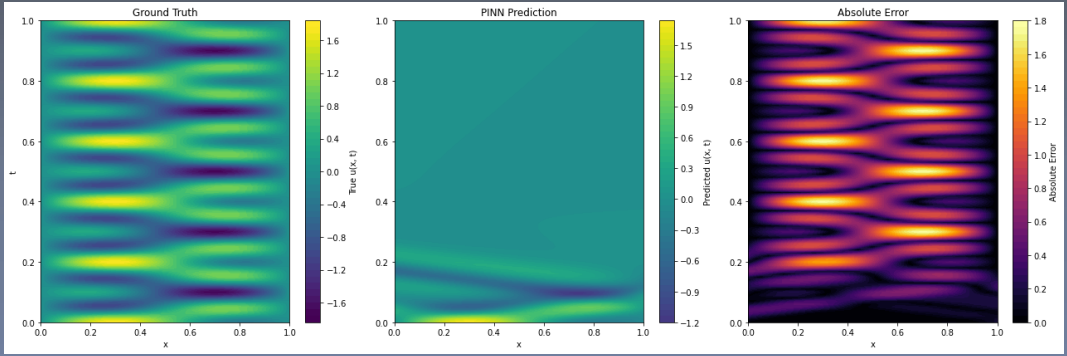
可视化：损失函数($\lambda_{i1} = \lambda_b = 100, \lambda_{i2} = 10$)

深度之眼
deepshare.net



可视化：预测值($\lambda_{i1} = \lambda_b = 100, \lambda_{i2} = 10$)

深度之眼
deepshare.net

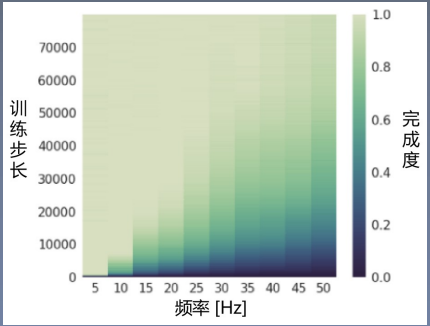


谱偏置

神经网络倾向于优先学习低频特征，而高频特征学习较慢。

关键观察: 高频成分需要更多的训练轮次才能收敛。对于 PINNs 的影响：

- 使多尺度问题的求解变得困难。
- 高频细节可能无法充分拟合。



参考文献：On the Spectral Bias of Neural Networks

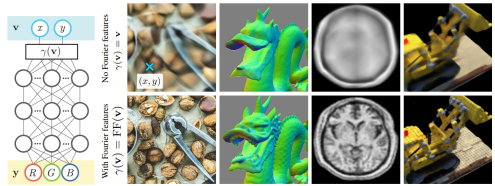
神经网络中的特征映射

深度之眼 deepshare.net

特征映射将输入空间转换为高维空间。有助于捕捉数据中的复杂模式和交互。

常见策略：

- **傅里叶特征映射**: 将输入添加正弦和余弦分量。
- **多项式特征**: 包括高阶项。



(a) Coordinate-based MLP

(b) Image regression $(x, y) \to$ RGB

(c) 3D shape regression $(x, y, z) \to$ occupancy

(d) MRI reconstruction $(x, y, z) \to$ density

(e) Inverse rendering $(x, y, z) \to$ RGB, density

Figure 1: Fourier features improve the results of coordinate-based MLPs for a variety of high-frequency low-dimensional regression tasks, both with direct (b, c) and indirect (d, e) supervision. We visualize an example MLP (a) for an image regression task (b), where the input to the network is a pixel coordinate and the output is that pixel's color. Passing coordinates directly into the network (top) produces blurry images, whereas preprocessing the input with a Fourier feature mapping (bottom) enables the MLP to represent higher frequency details.

参考文献： Fourier features let networks learn high frequency functions in low dimensional domains

傅里叶特征映射

深度之眼 deepshare.net

定义: 将输入 x 映射为正弦特征:

$$\gamma(x) = [\sin(2\pi k_1 x), \cos(2\pi k_1 x), \ldots, \sin(2\pi k_n x), \cos(2\pi k_n x)] \tag{1}$$

优势:

- 提高高频特征的表达能力。
- 减弱谱偏置的影响。

在 PINNs 中的应用:

- 加速多尺度 PDE 的收敛。
- 改善对尖锐梯度的表达能力。

时空多尺度PINNs

深度之眼
deepshare.net

S. Wang, H. Wang and P. Perdikaris *Computer Methods in Applied Mechanics and Engineering 384 (2021) 113938*

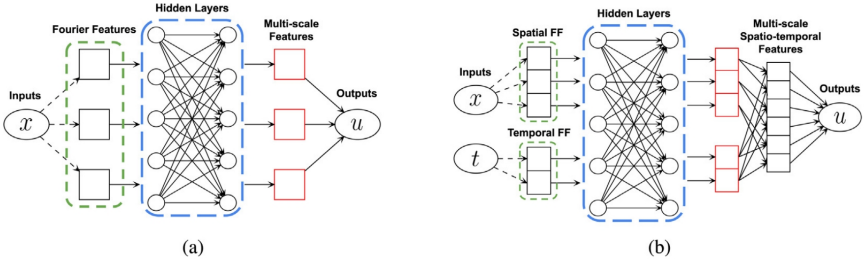


Fig. 8. *(a) Multi-scale Fourier feature architecture:* multiple Fourier feature embeddings (initialized with different σ) are applied to input coordinates and then passed through the same fully-connected neural network, before the outputs are finally concatenated with a linear layer. *(b) Spatio-temporal multi-scale Fourier feature architecture:* Multi-scale Fourier feature architecture: multiple Fourier feature embeddings (initialized with different σ) are separately applied to spatial and temporal input coordinates and then passed through the same fully-connected neural network. Merging of the spatial and temporal outputs is performed using a point-wise multiplication layer, before obtaining the final outputs through a linear layer.

- Wang, Sifan, Hanwen Wang, and Paris Perdikaris. "On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks."

多尺度Fourier特征网络：数学表达

深度之眼
deepshare.net

对于输入坐标$\boldsymbol{x}$: $\gamma^{(i)}(\boldsymbol{x}) = \begin{bmatrix} \cos(2\pi \boldsymbol{B}^{(i)}\boldsymbol{x}) \\ \sin(2\pi \boldsymbol{B}^{(i)}\boldsymbol{x}) \end{bmatrix}$, 对于$i = 1, 2, \ldots, M$

$\boldsymbol{H}_1^{(i)} = \phi(\boldsymbol{W}_1 \cdot \gamma^{(i)}(\boldsymbol{x}) + \boldsymbol{b}_1)$, 对于$i = 1, 2, \ldots, M$

$\boldsymbol{H}_\ell^{(i)} = \phi(\boldsymbol{W}_\ell \cdot \boldsymbol{H}_{\ell-1}^{(i)} + \boldsymbol{b}_\ell)$, 对于$\ell = 2, \ldots, L, i = 1, 2, \ldots, M$

$\boldsymbol{f}_\theta(\boldsymbol{x}) = \boldsymbol{W}_{L+1} \cdot \left[\boldsymbol{H}_L^{(1)}, \boldsymbol{H}_L^{(2)}, \ldots, \boldsymbol{H}_L^{(M)}\right] + \boldsymbol{b}_{L+1}$

其中:

- $\boldsymbol{B}^{(i)} \in \mathbb{R}^{m \times d}$中的每个元素从$\mathcal{N}(0, \sigma_i)$采样
- σ_i的典型值可以是$1, 20, 50, 100$等
- 权重和偏置与标准全连接神经网络相同

时空多尺度Fourier特征网络

深度之眼 deepshare.net

适用于时空多尺度问题的扩展架构:

$$\gamma_{\boldsymbol{x}}^{(i)}(\boldsymbol{x}) = \begin{bmatrix} \cos(2\pi \boldsymbol{B}_{\boldsymbol{x}}^{(i)} \boldsymbol{x}) \\ \sin(2\pi \boldsymbol{B}_{\boldsymbol{x}}^{(i)} \boldsymbol{x}) \end{bmatrix}, \quad \boldsymbol{H}_{\boldsymbol{x},1}^{(i)} = \phi(\boldsymbol{W}_1 \cdot \gamma_{\boldsymbol{x}}^{(i)}(\boldsymbol{x}) + \boldsymbol{b}_1) \tag{2}$$

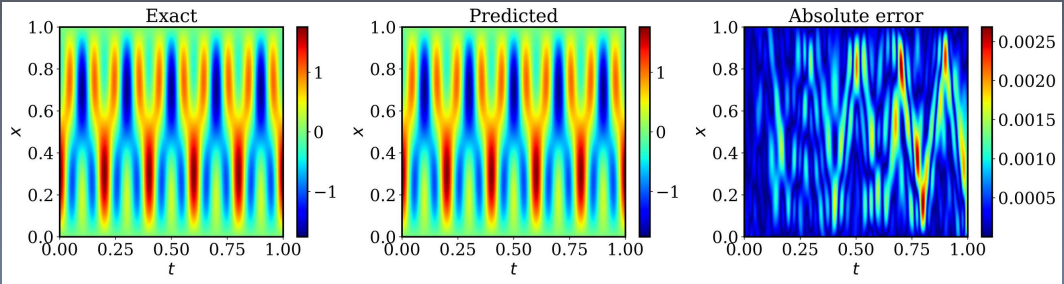
$$\gamma_{t}^{(j)}(t) = \begin{bmatrix} \cos(2\pi \boldsymbol{B}_{t}^{(j)} t) \\ \sin(2\pi \boldsymbol{B}_{t}^{(j)} t) \end{bmatrix}, \quad \boldsymbol{H}_{t,1}^{(j)} = \phi(\boldsymbol{W}_1 \cdot \gamma_{t}^{(j)}(t) + \boldsymbol{b}_1) \tag{3}$$

$$\boldsymbol{H}_{L}^{(i,j)} = \boldsymbol{H}_{\boldsymbol{x},L}^{(i)} \odot \boldsymbol{H}_{t,L}^{(j)} \quad (\text{点乘}) \tag{4}$$

$$\boldsymbol{f}_{\theta}(\boldsymbol{x}, t) = \boldsymbol{W}_{L+1} \cdot \left[\boldsymbol{H}_{L}^{(1,1)}, \ldots, \boldsymbol{H}_{L}^{(M_x, M_t)} \right] + \boldsymbol{b}_{L+1} \tag{5}$$

其中$\boldsymbol{B}_{\boldsymbol{x}}^{(i)}$和$\boldsymbol{B}_{t}^{(j)}$分别从$\mathcal{N}(0, \sigma_i^x)$和$\mathcal{N}(0, \sigma_j^t)$采样。

案例研究: 波动方程$c = 10$



https://github.com/PredictiveIntelligenceLab/MultiscalePINNs/tree/main

使用PINNs求解多尺度PDE

深度之眼
deepshare.net

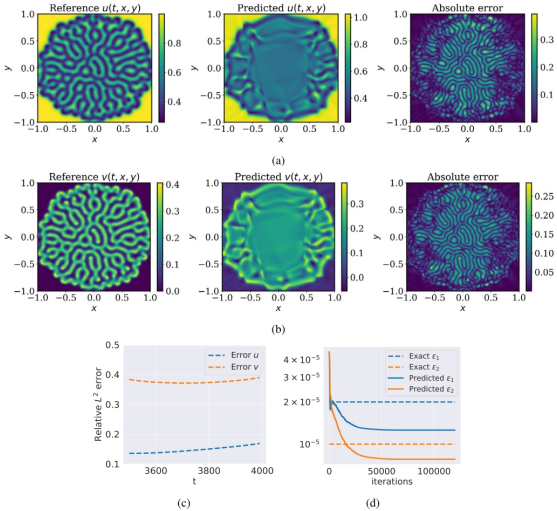
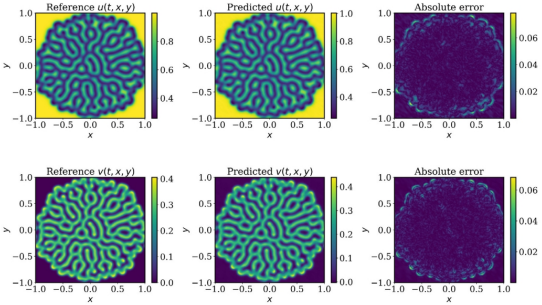


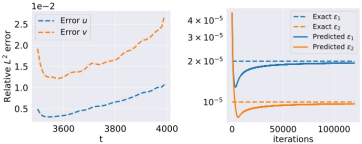
Fig. 18. *2D Gray–Scott equation:* Results obtained by training a fully-connected network (9 layers, 100 hidden units, tanh activations) after 120,000 iterations of gradient descent. (a)(b) Numerical estimations versus the predicted concentration fields u, v respective at the final

使用PINNs+FFN求解多尺度PDE

深度之眼
deepshare.net



(b)



(c) (d)

Fig. 17. *2D Gray–Scott equation:* Results obtained by training a fully-connected network (9 layers, 100 hidden units, tanh activations) with the spatio-temporal Fourier feature mappings after 120,000 iterations of gradient descent. (a) (b) Numerical estimations versus the predicted concentration fields u, v respective at the final time $t = 4000$. (c) Relative L^2 errors between the model predictions and the corresponding

深度之眼
deepshare.net

Thank you